



(19) 대한민국특허청(KR)
(12) 등록특허공보(B1)

(45) 공고일자 2021년05월24일
(11) 등록번호 10-2254585
(24) 등록일자 2021년05월14일

(51) 국제특허분류(Int. Cl.)
G06F 9/52 (2018.01) G06F 9/30 (2018.01)
G06F 9/38 (2006.01)
(52) CPC특허분류
G06F 9/52 (2013.01)
G06F 9/3004 (2013.01)
(21) 출원번호 10-2015-7035775
(22) 출원일자(국제) 2014년05월21일
심사청구일자 2019년04월17일
(85) 번역문제출일자 2015년12월17일
(65) 공개번호 10-2016-0010580
(43) 공개일자 2016년01월27일
(86) 국제출원번호 PCT/FI2014/050391
(87) 국제공개번호 WO 2014/188073
국제공개일자 2014년11월27일
(30) 우선권주장
13168732.9 2013년05월22일
유럽특허청(EPO)(EP)
(56) 선행기술조사문헌
KR1020110083941 A

(73) 특허권자
테크놀로지안 투트키무스케스쿠스 브이티티 오와
이
핀란드, 에스푸 02150, 테크니칸티 21
(72) 발명자
포르셀, 마르티
핀란드, 에프아이-90100 울루 울루, 테르바레이티
2 에이 17
(74) 대리인
김순웅

전체 청구항 수 : 총 15 항

심사관 : 지정훈

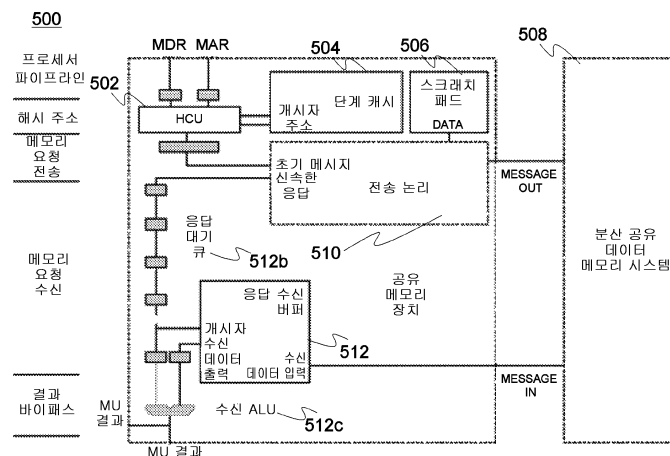
(54) 발명의 명칭 **예물레이션된 공유 메모리 구조를 위한 메모리 장치**

(57) 요약

예물레이션된 공유 메모리(ESM) 구조에서 인터리브드 인터-스레드 파이프라인을 구비하는 멀티-스레드화된 프로세서의 데이터 메모리의 참조를 처리하기 위한 메모리 장치(500)가 제공되며, 상기 메모리 장치는, 멀티스레드화된 실행의 진행중인 단계의 마지막까지 데이터 어레이가 유효한 상태로 유지되는 연상 캐시 메모리 어레이를 형

(뒷면에 계속)

대표도 - 도5



성하는 단계 캐시(504), 상기 메모리 어레이는 데이터 필드를 갖는 다수의 캐시 라인을 포함하고, 각각의 라인은, 바람직하게는 독점적으로, 주소 태그에 대한 데이터 필드 및 주소 태그에 의해 지정된 데이터 메모리 위치를 나타내는 제 1 스레드 id에 대한 데이터 필드를 포함하고, 및 중간 결과와 같은 멀티연산의 내부 데이터를 저장하기 위한 메모리 버퍼를 형성하는 스크래치패드(506)를 포함하고, 상기 버퍼는, 바람직하게는 독점적으로, 프로세서의 각각의 스레드에 대해 하나의 데이터 필드를 포함하고, 상기 메모리 장치는 캐시 검색을 위해 단계 캐시에 액세스하고, 프로세서에 의한 데이터 메모리(508)의 참조를 포함하는 멀티연산의 실행시 프로세서 파이프 라인의 다양한 클럭 사이클 및 다양한 단계에서 상기 내부 데이터를 검색 및/또는 저장하기 위해 스크래치패드(506)에 액세스하도록 구성된다. 메모리 참조를 처리하기 위한 대응하는 방법이 또한 제공된다.

(52) CPC특허분류

G06F 9/3851 (2013.01)

명세서

청구범위

청구항 1

에물레이션된 공유 메모리(ESM) 구조에서 인터리브드 인터-스레드 파이프라인을 구비하는 멀티-스레드화된 프로세서의 데이터 메모리(508)의 참조를 처리하기 위한 메모리 장치(500)에 있어서, 상기 메모리 장치는,

멀티스레드화된 실행의 진행중인 단계의 마지막까지 데이터 어레이가 유효한 상태로 유지되는 연상 캐시 메모리 어레이를 형성하는 단계 캐시(504), 상기 메모리 어레이는 데이터 필드를 갖는 다수의 캐시 라인을 포함하고, 각각의 라인은, 바람직하게는 독점적으로, 주소 태그에 대한 데이터 필드 및 주소 태그에 의해 지정된 데이터 메모리 위치를 나타내는 제 1 스레드 id에 대한 데이터 필드를 포함하고, 및

중간 결과와 같은 멀티연산의 내부 데이터를 저장하기 위한 메모리 버퍼를 형성하는 스크래치패드(506)를 포함하고, 상기 버퍼는, 바람직하게는 독점적으로, 프로세서의 각각의 스레드에 대해 하나의 데이터 필드를 포함하고,

상기 메모리 장치는 캐시 검색을 위해 단계 캐시(504)에 액세스하고, 프로세서에 의한 데이터 메모리(508)의 참조를 포함하는 멀티연산의 실행시 프로세서 파이프라인의 다양한 클럭 사이클 및 다양한 단계에서 상기 내부 데이터를 검색 및/또는 저장하기 위해 스크래치패드(506)에 액세스하도록 구성되는 것을 특징으로 하는 메모리 장치.

청구항 2

제 1 항에 있어서,

상기 메모리 장치는, 프로세서의 스레드에 의한 데이터 메모리의 참조에 대응해서, 참조된 메모리 주소와 일치하는 주소가 내부에 저장된 주소 태그를 근거로 발견되는지를 결정하기 위해 단계 캐시에 액세스하고, 이러한 경우, 일치하는 주소의 단계 캐시 라인의 스레드 id의 데이터 필드에서 스레드 id를 검색하고, 반대로, 캐시 검색이 오류인 경우, 소정의 데이터 교체 정책을 사용하여 단계 캐시 내의 참조된 메모리 주소에 해당하는 현재 스레드와 주소 태그의 스레드 id를 저장하도록 구성되는 것을 특징으로 하는 메모리 장치.

청구항 3

제 1 항 또는 제 2 항에 있어서,

상기 메모리 장치는 스레드에 대해 수신된 메모리 참조 데이터를 저장하기 위한 응답 수신 버퍼(512)를 더 포함하고, 상기 버퍼는 공유 메모리 시스템으로부터 들어오는 응답 메시지를 수신하도록 구성되는 것을 특징으로 하는 메모리 장치.

청구항 4

제 3 항에 있어서,

상기 응답 수신 버퍼(512)는 멀티포트 버퍼이고, 두 개의 연속적인 단계와 관련된 스레드에 대해 보유 공유 메모리 참조를 나타내는 데이터 및 보류 필드를 수용하기 위해 두 개의 데이터 어레이와 두 개의 포트를 포함하는 것을 특징으로 하는 메모리 장치.

청구항 5

제 1 항 또는 제 2 항에 있어서,

상기 메모리 장치는 메모리 쓰기 또는 읽기 연산과 관련해서 단계 캐시 검색이 오류인 경우 각각 응답 수신 버퍼(512)를 통해 참조된 데이터를 내부에 저장하거나 검색하기 위한 공유 메모리 시스템(508)으로 메모리 참조를 전송하고, 쓰기 연산과 관련해서 단계 캐시가 검색된 경우 쓰기 연산을 무시하며, 읽기 연산과 관련해서 단계 캐시가 검색된 경우 검색되거나 저장된 스레드 id와 함께 응답 수신 버퍼(512)로부터 결과를 획득하도록 구성되는 것을 특징으로 하는 메모리 장치.

청구항 6

제 1 항 또는 제 2 항에 있어서,

상기 메모리 장치는 멀티 연산과 관련해서 스크래치패드(506)를 액세스하고, 즉, 스크래치패드에 데이터를 저장하고 스크래치패드에서 데이터를 검색하며, 내부 프로세서 멀티연산 결과를 계산하기 위해 검색된 또는 저장된 스레드 id와 함께 이를 공유 메모리 시스템(508)에 전송하도록 구성되는 것을 특징으로 하는 메모리 장치.

청구항 7

제 1 항 또는 제 2 항에 있어서,

상기 메모리 장치는 멀티-프리픽스와 같은 최종 멀티연산 결과를 결정하기 위해 공유 메모리 시스템(508)으로부터 응답 수신 버퍼(512)에서 수신된 데이터 및 내부 프로세서 결과 모두를 이용하도록 구성되는 것을 특징으로 하는 메모리 장치.

청구항 8

제 1 항 또는 제 2 항에 있어서,

상기 단계 캐시에 액세스하도록 구성되는 해시 및 구성 장치(502)를 포함하고, 상기 해시 및 구성 장치는 바람직하게 액세스 포트를 통해, 더욱 바람직하게는 해시 및 구성 장치의 전용으로 구성되는 단일 액세스 포트를 통해 상기 단계 캐시에 결합되는 것을 특징으로 하는 메모리 장치.

청구항 9

제 8 항에 있어서,

상기 해시 및 구성 장치는 프로세서에 의한 메모리 연산, 메모리 주소, 메모리 데이터, 단계 카운터의 소정 수의 최하위 비트(least significant bit, LSB), 단계 캐시 액세스의 결과로 이루어진 군에서 선택되는 적어도 하나의 요소를 기반으로 메모리 참조 메시지를 구성하도록 구성되는 것을 특징으로 하는 메모리 장치.

청구항 10

제 1 항 또는 제 2 항에 있어서,

상기 단계 캐시(504)는 멀티-웨이 집합 연산(multi-way set associative) 단계 캐시인 것을 특징으로 하는 메모리 장치.

청구항 11

제 1 항 또는 제 2 항에 있어서,

상기 단계 캐시 내의 주소는 랜덤하게 선택된 해싱 함수를 이용하여 해싱되는 것을 특징으로 하는 메모리 장치.

청구항 12

제 1 항 또는 제 2 항에 있어서,

스크래치패드(506)에 액세스하도록 구성되는 메모리 요청 전송 논리 개체(510)를 포함하고, 상기 스크래치패드는 액세스 포트를 통해, 바람직하게는 전송 논리 개체(510)의 전용으로 구성되는 단일 액세스 포트를 통해 액세스될 수 있는 것을 특징으로 하는 메모리 장치.

청구항 13

제 12 항에 있어서,

상기 전송 논리 개체는 공유 메모리 시스템에 메모리 참조를 전송하고, 소정의 멀티연산의 중간 결과 또는 내부 프로세서 결과를 결정하며, 바람직하게는 메모리 응답 대기 큐를 통해 응답 수신 버퍼(512)에 대해 로컬 응답으로서 내부 메모리 (멀티)연산의 결과와 같은 내부 데이터를 제공하도록 구성되는 것을 특징으로 하는 메모리 장치.

청구항 14

제 1 항 또는 제 2 항에 있어서,

소정 수의 파이프라인 단계와 관련된 응답 대기 큐(512b)를 포함하고, 상기 큐는, 예를 들어, 전송 논리로부터의 로컬 응답이 응답 수신 버퍼에 대해 적절한 순서로 전달되어 공유 메모리로부터 수신된 관련 메모리 참조를 적시에 검색한 후 수신 ALU(512c)를 이용하여 최종 연산 결과를 도출할 수 있도록, 메모리 요청 전송 논리 객체와 응답 수신 버퍼를 연결하는 것을 특징으로 하는 메모리 장치.

청구항 15

에플레이션된 공유 메모리(ESM) 구조에서 인터리브드 인터-스레드 파이프라인을 구비하는 멀티-스레드화된 프로세서의 데이터 메모리의 참조를 처리하기 위한 방법에 있어서, 상기 방법은,

멀티스레드화된 실행의 진행중인 단계의 마지막까지 데이터 어레이가 유효한 상태로 유지되는 연상 캐시 메모리 어레이를 형성하는 단계 캐시(504)를 획득하는 단계, 상기 메모리 어레이는 데이터 필드를 갖는 다수의 캐시 라인을 포함하고, 각각의 라인온, 바람직하게는 독립적으로, 주소 태그에 대한 데이터 필드 및 주소 태그에 의해 지정된 데이터 메모리 위치를 나타내는 제 1 스레드의 스레드 id에 대한 데이터 필드를 포함하고,

중간 결과와 같은 멀티연산의 내부 데이터를 저장하기 위한 메모리 버퍼를 형성하는 스크래치패드(506)를 획득하는 단계, 상기 버퍼는, 바람직하게는 독립적으로, 프로세서의 각각의 스레드에 대해 하나의 데이터 필드를 포함하고,

캐시 검색을 위해 상기 단계 캐시에 액세스하고, 프로세서에 의한 데이터 메모리의 참조를 포함하는 멀티연산의 실행시 프로세서 파이프라인의 다양한 클록 사이클 및 다양한 단계에서 상기 내부 데이터를 검색 및/또는 저장하기 위해 스크래치패드에 액세스하는 단계를 포함하는 것을 특징으로 하는 방법.

발명의 설명

기술 분야

일반적으로, 본 발명은 컴퓨터 과학 및 컴퓨터 구조에 관한 것이다. 특히, 본 발명은 에플레이션된 공유 메모리 구조와 관련하여 메모리 액세스 및 멀티(-프릭스)연산(multi(-prefix)operation)의 최적화에 관한 것이다.

[0001]

배경 기술

- [0002] 공유 메모리 구조(shared memory architecture, SMA)에서, 데이터와 프로그램 분할은 일반적으로 다수의 스레드(thread)에 의한 처리를 필요로 하는 데이터를 공유 메모리에 배치하고 프로그램을 프로세서에 대해 더욱 독립적으로 분할함으로써 수행되고, 따라서 처리가 항상 국부적으로 발생하고 이에 따라 프로그래머가 데이터를 이동시켜야 하는 메시지 패싱 구조(message passing architectures, MPA)에 비해 프로그래밍을 더욱 용이하게 한다. 불행하게도, 대부분의 SMA는 캐시 일관성(및 이에 따른 지연 허용(latency tolerance)) 및 동시성(synchronicity) 유지에 비용이 많이 드는 다수의 상호 연결된 프로세서-캐시 쌍으로 구성되는 분산 공유 메모리 구조를 사용한다. 이는 통신 집중 문제에서 이들의 성능을 심지어 엉망으로 만들 수 있다.
- [0003] 예를 들어, 상기한 문제를 해결하기 위해, 에뮬레이션된 공유 메모리(emulated shared memory, ESM) 또는 공유 메모리 에뮬레이션 구조가 도입되었다. 이들은 대용량 통신망을 통해 일반적으로 균일하고 동시에 액세스 가능한 공유 메모리에 연결되는 일련의 멀티스레드 프로세서를 포함한다. 메모리 시스템에서의 지연은 진행중인 메모리 참조를 중복시킴으로써 은폐되고, 기계 명령어 수준에서 동시성을 보장하는 특수 저비용 동기화 메커니즘이 확립된다. ESM 시스템은 실제 하드웨어 구조가 물리적으로 분산된 메모리를 포함하더라도 사용자에게 이상적인 공유 메모리의 인식을 제공한다. 이론적인 관점에서, 이들 구조는 일반적으로 단순성과 표현성으로 인해 계산 문제의 본질적인 병렬성(parallelism)뿐만 아니라 병렬 알고리즘의 성능 및 실행 비용을 설명하고 분석하기 위한 모델로서 사용되는 추상적인 병렬 랜덤 액세스 머신(parallel random access machine, PRAM)을 에뮬레이션하려고 한다. PRAM 모델은 일반적으로 동일한 클록 하에 작동하는 일련의 프로세서 및 이들에 연결된 균일한 단일 단계의 액세스 가능한 공유 메모리를 말한다.
- [0004] 따라서, ESM은 기계 명령어의 실행에서 목시적인 동기화, 효율적인 지연 은폐 기술, 및 과중한 랜덤 및 동시 액세스 작업량에서도 모든 메모리 참조를 라우팅하는 충분한 대역폭을 제공하기 때문에 칩 멀티프로세서(chip multiprocessor, CMP)의 프로그래밍 가능성 및 성능 확장성 문제를 해결하는 실현 가능한 기술이다. 프로그래머가 각각의 전역 메모리 액세스 이후 명백하게 실행의 스레드를 동기화할 필요는 없지만 이를 자동으로 처리하는 하드웨어에 의존할 수 있는 반면, 예를 들어, 메시지 패싱 구조(MPA)에서, 프로그래머는 통신을 명백하게 정의하고, 서브태스크(subtask)를 동기화하며, MPA의 프로그래밍을 어렵게 하는 스레드 간에 데이터 및 프로그램 분할을 설명해야 하기 때문에 동시 실행은 프로그래밍을 더욱 용이하게 하는 것으로 간주된다. 공유 메모리 에뮬레이션에서 사용되는 지연 은폐는 대용량 계산 기법을 이용하고, 여기서 하나의 스레드가 전역 공유 메모리를 참조하는 동안 그 밖의 스레드가 실행된다. 대용량 계산 기법은 사용 가능한 스레드 레벨의 병렬성으로부터 추출된 병렬 여유성(parallel slackness)을 사용하기 때문에, 스누핑(snooping) 또는 디렉토리 기반의 캐시 일관성 메커니즘에 의존하고 따라서 제한된 대역폭 또는 디렉토리 액세스 지연 및 과중한 일관성 트래픽 유지에 시달리는 종래의 대칭적 멀티프로세서와 불균일 메모리 액세스(non-uniform memory access, NUMA) 시스템에 비해 향상된 확장성을 제공하는 것으로 간주된다.
- [0005] 최근, 특정한 경우에 대수 인자에 의해 프로그래밍을 더욱 단순화하고 성능을 향상시키는, PRAM의 동시 읽기 동시 쓰기(concurrent read concurrent write, CRCW) 메모리 액세스의 변형을 구현하기 위해 단계 캐시(step cache)를 통합하는 확장형 ESM 구조가 제안되었다. 또한, PRAM 모델의 더욱 강력한 멀티연산 동시 읽기 동시 쓰기(MCRCW) 변형을 구현하는, 일정한 실행 시간 멀티(-프리픽스)연산을 지원하기 위한 메커니즘이 단계 캐시의 연관성을 결합하기 위해 단계 캐시에 부착되는 스크래치패드의 도움으로 구현되었다. 예를 들어, 간행물 1(M. Forsell, Step Caches-공유 메모리 MP-SOCs 상의 동시 메모리 액세스에 대한 새로운 접근법, 제 23차 IEEE NORCHIP 학술회의, 2005년 11월 21-22일, 2005, 오울루, 핀란드, 74-77), 간행물 2(M. Forsell, CRCW 연산에서 단계 캐시의 연관성과 크기의 축소, 병렬 및 분산 계산 모델에서의 진전에 대한 제 8차 워크숍(제 20차 IEEE 국제 병렬 및 분산 처리 학술 대회와 함께), 간행물 3(M. Forsell, 단계 캐시 MP-SOCs에 대한 멀티연산의 실현, 시스템-온-칩 2006(SOC'06) 국제 학술 대회, 2006년 11월 14-16일, 2006, 탐페레, 핀란드, 77-82), 간행물 4(M. Forsell, TOTAL ECLIPSE-병렬 랜덤 액세스 머신의 효율적인 구조 실현, Alberto Ros가 편집한 병렬 및 분산 계산, IN-TECH, 비엔나, 2010, 39-64) 및 간행물 5(M. Forsell 및 J. Roivainen, 에뮬레이션된 공유 메모리 CMPs에서 멀티프리픽스 연산의 지원, 병렬 및 분산 처리 기술 및 응용 2011년 국제 학술 대회(PDPTA'11), 20011년 7월 18-21일, 라스베이거스, USA, 506-512)는 이러한 솔루션의 다양한 양태를 고려하고 따라서 그 전체가 본원에 참고로 포함된다. 멀티(-프리픽스)연산은, 예를 들어, ADD, SUB, MAX 등과 같은 많은 기본 연산에 대해 정의될 수 있고 병렬 알고리즘을 표현할 수 있는 기능으로 인해 병렬 기본형으로 간주될 수 있다. 이들은 스레드를 동시에 실행하는 구조의 경쟁 조건 및 그 밖의 모순 없이 여러 개의 프로세서에 의해 동시에 액세스되는 동

기화 및 병렬 데이터 구조에 대해 사용될 수 있다.

[0006] 도 1에, 실리콘 플랫폼 상에서 공유 메모리를 에뮬레이션하기 위한 확장형 구조의 고차원 도면이 도시되어 있다. 이는 물리적으로 분산되었지만, 물리적으로 확장 가능한 높은 대역폭의 상호 연결 네트워크(108)를 통해 논리적으로 공유된 (데이터) 메모리(M1, M2, M3, ..., Mp (112))에 연결된 일련의 프로세서(코어)(P1, P2, P3, ..., Pp (102))를 포함한다. 데이터 메모리(112)와 관련된 활성 메모리 장치(110)는 메모리 참조를 처리하기 위해 이용되는 메모리 제어 논리 장치로 간주될 수 있다. 활성 메모리 장치(110)는, 예를 들어, 멀티(-프리픽스)연산시 다수의 메모리 참조가 동일한 메모리 위치를 대상으로 하는 경우와 관련된 계산을 관리하도록 구성된다. 명령어 메모리 모듈(I1, I2, I3, ..., Ip (104))은 각각의 프로세서(102)에 대한 프로그램 코드를 저장하도록 구성된다. 분산 메모리 기반의 구현에 의해 공유 메모리를 효율적으로 에뮬레이션하기 위해, 프로세서(102)는 Tp-단계 사이클의 인터리브드(interleaved) 인터-스레드 파이프라인($T_p \geq$ 네트워크의 평균 지연)을 이용하여 멀티스레드화된다. PRAM 모델은 파이프 라인의 전체 사이클이 일반적으로 단일 PRAM 단계에 해당하도록 구조에 결합된다. 멀티스레드화된 실행의(전체 파이프라인, 즉, 실제 실행 단계를 포함하는 모든 파이프 라인의 단계에 대한) 한 단계 동안, CMP의 각각의 프로세서의 각각의 스레드는 최대 하나의 공유 메모리 참조 하위 명령어를 포함하는 명령어를 실행한다. 따라서, 한 단계는 다수, 적어도 $T_p + 1$ 의 클럭 사이클 동안 지속된다.

[0007] 도시된 구조에서, 단계 캐시는 일반적으로 연상(associative) 메모리 버퍼이고 여기서 데이터는 멀티스레드화된 실행의 진행중인 단계의 마지막까지만 유효한 상태로 유지된다. 동시 액세스에 대한 단계 캐시의 주요 기여는 이들이 각각의 참조된 메모리 위치에 대한 첫 번째 참조를 제외한 모든 것을 단계적으로 필터링한다는 것이다. 이는 P T_p 에서 P까지 위치당 요청의 수를 줄임으로써, $T_p \geq P$ 라고 가정할 때 이들이 단일 포트의 메모리 모듈 상에서 순차적으로 처리될 수 있게 한다. 스크래치패드는, 단계 캐시 및 실제 내부 프로세서와 멀티연산용 내부 프로세서의 계산을 처리하는 최소 온-코어 및 오프-코어 산술 논리 장치(arithmetic logic unit, ALU)의 도움으로 멀티연산을 구현하는데 있어서 단계 캐시의 연관성을 제한되게 유지하기 위해 메모리 액세스 데이터를 저장하기 위해 사용되는 주소화(addressable) 메모리 버퍼이다. 스크래치패드는 단계 캐시와 결합하여 소위 스크래치패드 단계 캐시 단위(S1, S2, S3, ..., Sp (106))를 확립할 수 있다.

[0008] 검토한 솔루션의 한 가지 근본적인 사상은 실제로 인터리브드 방식으로 효율적으로 실행되고 네트워크의 지연을 은폐하는 일련의 스레드로 각각의 프로세서 코어(102)를 할당하는 것에 있다. 스레드가 메모리 참조를 형성함에 따라, 실행되는 스레드는 변경되고 다음 스레드가 이의 메모리 요청 등을 할 수 있다. 스레드가 다시 실행되기 전에 메모리 참조의 응답이 프로세서 코어로 도착되면 메모리 지연은 발생하지 않을 것이다. 이는 네트워크의 대역폭이 매우 충분해야 하고, 파이프라인 메모리 액세스 트래픽 내에 핫스팟이 방지될 수 있는 것을 필요로 한다. 연속적인 명령어 간의 동시성은, 예를 들어, 단계 사이에서 탄력 있는 동기화 파장을 사용함으로써 보장될 수 있다.

[0009] 도 2는, 예를 들어, 데이터 메모리 모듈(112) 및 스크래치패드(206B)와 관련된 상기한 활성 메모리 장치(112)(ALU 및 인출기(fetcher)와 함께)를 포함하는 ESM CMP 구조(200)의 일례를 도시하고 있다. 네트워크(108)는 스위치(108B)를 갖는 고대역 파이프라인 메모리 시스템으로 작용하는 메쉬형 상호 연결 네트워크일 수 있다. 메모리 액세스 지연은 스레드가 네트워크(108)를 통해 균일하게 액세스 가능한 분산 공유 메모리를 참조하는 동안 다른 스레드를 참조함으로써 은폐된다. 통신 중인 참조와 핫스팟의 혼잡은 높은 대역폭(양분 $BW \geq P/4$)과 분산 메모리 모듈 상의 메모리 위치의 무작위 해싱을 특징으로 하는 효율적인 무교착(dead-lock free) 상호 연결 구조로 방지된다. 명령어의 실행은 각각의 스레드가 하나의 명령어를 실행하는 동안 단일 PRAM 단계에 해당하는 단계에서 발생한다.

[0010] 상기한 많은 장점에도 불구하고, ESM 시스템은 진정으로 최적의 방식으로 실현하기 어려운 것으로 보였다. 강력한 동시 메모리 액세스와 멀티(-프리픽스)연산을 지원하기 위해 단계 캐시와 스크래치패드 기술을 사용하는 물리적으로 가능한 메모리 장치(MU)를, 기본적으로 구성 가능한 ESM인 칩 멀티프로세서 구조의 성능 및 프로그래밍 가능성 제한의 제거(REmoving Performance and programmability Limitations of Chip multiprocessor Architectures, REPLICA)와 같은 매우 효과적인 에뮬레이션된 공유 메모리 구조의 하나의 핵심 성분으로 쉽게 이해할 수 있다. 이러한 MU는 공유 메모리 시스템에 진행중인 메모리 참조를 전송할 뿐만 아니라 이로부터 가능한 응답을 대기하고 수신한다. 불행하게도, 아래에서 더욱 상세하게 설명되는 종래 기술의 MU 솔루션에서, 낮은 수준의 구현 세부사항은 존재하지 않고 제시된 구성은 비교적 복잡한 멀티포트 단계 캐시와 스크래치패드 또는 복잡한 분류 배열 및 대형 노드 방식 버퍼를 필요로 한다. 또한, 종래 솔루션의 수신 논리는 하나의 클럭 사이클 동안 단계 캐시와 스크래치패드 모두를 액세스하고, 후자의 성능은 모든 메모리 연산 동안 두 단계의 최소

지연에 의해 악화된다. 이상은 모두 이미 사용 가능한 MU 솔루션을 오히려 비현실적이게 한다.

- [0011] 도 3은 단계 캐시를 사용하는 전형적인 MCRCW ESM 프로세서(300)의 고차원 블록도 및 파이프라인을 나타내고 있다. 단계 캐시 기반의 MCRCW (C)ESM CMP 내의 프로세서는 A개의 ALU, M개의 메모리 장치(MU), 분산 또는 통합 레지스터 블록, 시퀀서 및 일부 글루 로직(glue logic)을 포함한다. 도면에서, Ax 302는 ALU x를 나타내고, IF 308은 명령어 인출 논리를 나타내고, MEM 304는 메모리 장치 단계, OS 306은 오퍼랜드(operand) 선택 논리를 나타내며, SEQ 310은 시퀀서를 나타낸다. 도면에 나타난 바와 같이, 메모리 장치 대기 세그먼트 이전 및 이후에 논리적으로 배치되는 ALU(302)가 있다.
- [0012] 도 4를 참고로, 종래의 메모리 장치(MU) 구조가 이하에서 더욱 상세하게 검토된다. 도시된 구조에서, (기준) MU는 해시 및 구성 장치(hash and compose unit, HCU, 402), 전송 논리(410), 이중 어레이 단계 캐시(404), 스크래치패드(406), 응답 수신 논리(412) 및 수신 ALU(414)를 포함한다.
- [0013] HCU(402)는 메모리 참조에 대한 해시 주소(hash address)를 계산하고, 데이터 메모리 레지스터(MDR)에 정해진 메모리 데이터, 메모리 주소 레지스터(MAR)에 정해진 메모리 어드레스, 스레드 식별자, 연산 레지스터로부터의 메모리 연산, 및 단계 카운터가 제공한 현재 단계의 LSB로부터 진행중인 메모리 참조 메시지를 구성한다.
- [0014] 메모리 (하위)명령어를 기반으로, 전송 논리(410)는 단계 캐시(404)와 스크래치패드(406)에 액세스하고 공유 메모리 시스템(408)에 해당 참조(메모리 요청)를 전송하거나 또는 스크래치패드의 데이터에 대한 내부 메모리 연산 결과를 계산하고 그 결과를 신속한 응답으로서 응답 수신 논리(412)에 직접 전송한다.
- [0015] 응답 수신 논리(412)는 공유 메모리 시스템과 전송 논리(410)에서 들어오는 메모리 요청 응답 메시지를 수신하도록 구성된다. 이는 이들 둘의 주소 필드를 비교하고 수신 로드 내의 스레드의 수신된 데이터 레지스터를 설정하려고 한다. 응답과 전송 로직 데이터 모두가 서로에 대해 이동하기 때문에 파이프라인 단계당 두 개의 비교기가 필요하다.
- [0016] 단계 캐시(404)(데이터가 멀티스레드화된 실행의 진행중인 단계의 마지막에 대해서만 유효한 상태로 유지됨)는 상기한 바와 같이 각각의 참조된 메모리 위치에 대한 첫 번째 참조를 제외한 모든 것을 프로세서 방식으로 그리고 단계적으로 필터링함으로써 캐시 일관성 문제 없이 공유 메모리(408)로의 CRCW 액세스를 구현하기 위해 사용된다. 각각의 캐시 라인은 바람직하게, 데이터, 개시자(initiator, 주소로 지정된 위치를 나타내는 제 1 스레드의 스레드 id), 및 주소 태그를 포함한다. MU 다른 단계에 속하는 스레드를 전달할 수 있는 파이프라인의 전송 및 수신 단계 모두로부터의 단계 캐시를 MU가 액세스할 필요가 있기 때문에 두 개의 캐시 어레이와 두 개의 포트가 필요하다.
- [0017] 프로세서가 메모리(408)를 참조할 때마다, 단계 캐시 검색이 수행된다. 라인이 사용 중이고, 주소가 해당 라인의 주소 태그와 일치하며, 참조의 현재 단계의 최하위 비트(least significant bit, LSB)가 해당 라인의 단계의 두 개의 LSB와 일치하는 경우 해당 캐시 라인에 대해 검색이 검출된다. 검색이 성공한 경우, 쓰기는 바로 무시되는 반면, 캐시로부터 데이터에 액세스함으로써 읽기가 바로 완료된다. 검색 오류의 경우, 참조는 교체 정책(replacement policy)을 사용하여 캐시에 저장되고 보류(읽기를 위해)로 표시된다. 캐시 라인에 참조 정보를 저장하는 것과 동시에, 참조 그 자체가 공유 메모리 시스템(408)으로 전송된다. 메모리 시스템(408)으로부터 읽기의 응답이 도착하면, 데이터는 참조 정보를 저장하고 있는 단계 캐시 라인의 데이터 필드로 입력되고, 보류 필드는 지워진다. 단계 필드가 다시 현재 단계의 최하위 비트와 일치하기 전에 라인을 무효화하기 위해 소정의 캐시 붕괴 논리(cache decay logic)가 이용된다.
- [0018] 별도의 이중 포트 프로세서 수준의 멀티연산 메모리인 제한된 연관성의 단계 캐시로 멀티(-프리픽스)연산을 구현하기 위해, 스크래치패드(406)가 사용된다. 참조에 대한 정보를 단계 캐시로부터 제거시킬 수 있는 가능한 충돌과는 무관하게 정보를 저장하는 저장소에 단계 캐시에 대한 각각의 멀티연산 시퀀스의 개시자 스레드의 id 및 내부 개시 스레드 id의 레지스터뿐만 아니라 참조 정보를 저장할 필요가 있기 때문이다. 스크래치패드(406)는 프로세서의 각각의 스레드에 대해 데이터, 주소 및 보류용 필드를 갖는다. 단계 캐시(404)에 쓰여질 데이터가 스크래치패드(406)에도 쓰여지고, 특정 위치를 나타내는 제 1 스레드의 id가 단계 캐시(404)와 개시자 레지스터(나머지 참조를 위해)에 저장되고, 멀티연산에 대한 보류 비트가 스크래치패드(406)에 유지되고, 응답 데이터가 스크래치패드(406)에 저장되며, 연산을 종료하기 위한 응답 데이터가 스크래치패드(406)로부터 검색되도록 두 개의 명령어의 시퀀스를 사용하여 멀티연산이 구현될 수 있다(아래의 표 1의 알고리즘 1 참조).

표 1

[0019]

알고리즘 1: 기준 MU에서 2-단계 MPADD 멀티연산의 구현

```

PROCEDURE Processor::Execute::BMPADD ( Write_data , Write_Address )
Search Write_Address from the StepCache and put the result in matching_index
IF not found THEN
    IF the target line pending THEN
        Mark memory system busy until the end of the current cycle
    ELSE Read_data := 0
        StepCache[matching_index].data := Write_data
        StepCache[matching_index].address := Write_Address
        StepCache[matching_index].initiator_thread := Thread_id
        ScratchPad[Thread_id].Data := Write_data
        ScratchPad[Thread_id].Address := Write_Address
        Initiator_thread:=Thread_id
    ELSE
        Read_data := StepCache[matching_index].data
        StepCache[matching_index].data:= Write_data + Read_data
        ScratchPad[Initiator_thread].Data := Write_data + Read_data
        Initiator_thread := StepCache[matching_index].Initiator_thread

PROCEDURE Processor::Execute::EMPADD ( Write_data , Write_Address )
IF Thread_id<>Initiator_thread THEN
    IF ScratchPad[Initiator_thread].pending THEN
        Reply_pending := True
    ELSE
        Read_data := Write_data + ScratchPad[Initiator_thread].Data
    ELSE
        IF Write_Address = ScratchPad[Initiator_thread].Address THEN
            Send a EMPADD reference to the memory system with
            - address = Write_Address
            - data = ScratchPad[Initiator_thread].Data
            ScratchPad[Thread_id].pending := True
        ELSE
            Commit a Multioperation address error exception

PROCEDURE Module::Commit_access::EMPADD ( Data , Address )
Temporary_data := Memory [Address]
Memory[Address] := Memory[Address] + Data
Reply_data := Temporary_data

PROCEDURE Processor::Receive_reply::EMPADD ( Data , Address , Thread )
Read_Data[Thread] := Data | Read_Data[Thread]+Data (if Thread?Thread_id)
ScratchPad[Thread].Data := Data
ScratchPad[Thread].Pending := False
ReplyPending[Thread_id] := False
FOR each successor of Thread DO
    IF ReplyPending[successor] THEN
        Read_data[successor] := Read_data[successor] + Data
        ReplyPending[successor] := False

```

[0020]

시작 연산(임의의 순서의 멀티프리픽스 연산을 위한 BMPxxx)은 외부 메모리 시스템을 참조하지 않고 단계 캐시 위치에 대해 프로세서 방식의 멀티연산을 실행한다. 종료 연산(임의의 순서의 멀티프리픽스 연산을 위한 EMPxxx)은 나머지 멀티연산을 수행함으로써 이전에 개시된 메모리 위치에 대한 제 1 참조가 오퍼랜드로서의 프

로세서 방식의 멀티연산 결과를 이용하여 외부 메모리 참조를 실행시키도록 한다. 동일한 위치를 대상으로 하는 외부 메모리 참조는 멀티연산의 유형에 따라 해당 메모리 모듈의 활성 메모리 장치에서 처리될 수 있고, 참여 프로세서의 스크래치패드로 다시 응답 데이터가 전송된다.

[0021] 동시 액세스 ESM 시스템의 대안적인 실현은 레너드의 알고리즘(Ranade's algorithm) 또는 실제 멀티포트 메모리의 캐시리스(cacheless) 변형을 사용한다: 레너드의 알고리즘은 물리적 분산 메모리 시스템 상의 메모리 참조의 분류 및 전달을 기반으로 공유 메모리 추상을 구현한다. 이 솔루션은 신속한 멀티프릭스 연산(MCRCW PRAM 모델)을 구현할 수 있는 능력과 같은 일부 장점을 제공하지만, 이는 또한 모든 액세스가 복잡한 하드웨어 분류기를 필요로 하는 연속적인 분류 및 액세스 단계 및 네트워크의 노드에 대한 결합된 참조를 저장하기 위한 버퍼를 포함하기 때문에 개선의 여지를 남긴다. 또한, 기존의 구조는 비확장형 상호 연결 토폴로지를 필요로 한다. 실제 멀티포트 메모리는 각각의 메모리 셀 어레이의 액세스 회로를 동시 메모리 포트의 수만큼 복제함으로써 메모리 셀 수준까지의 동시 액세스를 제공한다. 이는 포트의 수(셀의 크기는 오직 선형으로만 증가함)에 대해 이차적으로 액세스 회로가 차지하는 면적을 증가시키며, 포트의 수가 예를 들어 네 개를 초과하는 경우 이를 불가능하게 한다. 캐시 일관성은 효율적인 공유 메모리 알고리즘을 이용하여 유지하는 것이 적어도 멀티포트 메모리만큼 매우 비싸기 때문에, 캐시가 프로세서 다음에 위치하는 일반적인 캐시 기반 구조는 고급 공유 메모리 연산에서 사용될 수 없다.

발명의 내용

해결하려는 과제

[0022] 본 발명의 목적은 공지의 ESM 구성에 의해 아직 완전히 만족스럽게 해결되지 않은 상기한 하나 이상의 문제를 완화하고 내부의 개선된 메모리 액세스를 위한 실현 가능한 솔루션을 제공하는 것이다.

과제의 해결 수단

- [0023] 상기 목적은 본 발명에 따른 메모리 장치의 실시형태에 의해 달성된다.
- [0024] 따라서, 본 발명의 일 양태에서, 에뮬레이션된 공유 메모리(ESM) 구조에서 인터리브드 인터-스레드 파이프라인을 구비하는 멀티-스레드화된 프로세서의 데이터 메모리의 참조를 처리하기 위한 메모리 장치가 제공되고, 상기 메모리 장치는,
- [0025] 멀티스레드화된 실행의 진행중인 단계의 마지막까지 데이터 어레이가 유효한 상태로 유지되는 연상 캐시 메모리 어레이를 형성하는 단계 캐시, 상기 메모리 어레이는 데이터 필드를 갖는 다수의 캐시 라인을 포함하고, 각각의 라인, 바람직하게는 독점적으로, 주소 태그에 대한 데이터 필드 및 주소 태그에 의해 지정된 데이터 메모리 위치를 나타내는 제 1 스레드 id에 대한 데이터 필드를 포함하고, 및
- [0026] 중간 결과와 같은 멀티연산의 내부 데이터를 저장하기 위한 메모리 버퍼를 형성하는 스크래치패드를 포함하고, 상기 버퍼는, 바람직하게는 독점적으로, 프로세서의 각각의 스레드에 대해 하나의 데이터 필드를 포함하고,
- [0027] 상기 메모리 장치는 캐시 검색을 위해 단계 캐시에 액세스하고, 프로세서에 의한 데이터 메모리의 참조를 포함하는 멀티연산의 실행시 프로세서 파이프라인의 다양한 클럭 사이클 및 다양한 단계에서 상기 내부 데이터를 검색 및/또는 저장하기 위해 스크래치패드에 액세스하도록 구성된다.
- [0028] 일 실시형태에서, 상기 메모리 장치는, 프로세서의 스레드에 의한 데이터 메모리의 참조에 대응해서, 참조된 메모리 주소와 일치하는 주소가 내부에 저장된 주소 태그를 근거로 발견되는지를 결정하기 위해 단계 캐시에 액세스하고, 이러한 경우, 즉, 단계 캐시가 검색된 경우, 일치하는 주소의 단계 캐시 라인의 스레드 id의 데이터 필드에서 스레드 id를 검색하고, 반대로, 캐시 검색이 오류인 경우, 소정의 데이터 교체 정책을 사용하여 단계 캐시 내의 참조된 메모리 주소에 해당하는 현재 스레드와 주소 태그의 스레드 id를 저장한다.
- [0029] 또 다른, 바람직하게는 추가적인 실시형태에서, 상기 메모리 장치는 스레드에 대해 수신된 메모리 참조 데이터를 저장하기 위한 응답 수신 버퍼를 더 포함한다. 상기 버퍼는 공유 메모리 시스템으로부터 들어오는 (응답) 메시지를 수신하도록 구성된다.
- [0030] 상기 응답 수신 버퍼는 멀티포트 버퍼이고 두 개의 연속적인 단계와 관련된 데이터 및 예를 들어 "보류" 필드를 수용하기 위해, 예를 들어, 두 개의 데이터 어레이와 두 개의 포트를 포함할 수 있으며, 이에 따라 상기 버퍼는 적어도 함축적으로 단계-인식 또는 단계 지정 가능하다.

- [0031] 다양한 실시형태에서, 단계 데이터는 예를 들어 스레드별(thread-specific) 데이터 필드 또는 레지스터를 사용하여 저장될 수 있다. 또한 메모리 참조 메시지는 바람직하게 단계 데이터용 필드를 포함한다.
- [0032] 또 다른 실시형태에서, 상기 메모리 장치는 메모리 쓰기 또는 읽기 연산과 관련해서 단계 캐시 검색이 오류인 경우 각각 응답 수신 버퍼를 통해 참조된 데이터를 내부에 저장하거나 검색하기 위한 공유 메모리 시스템으로 메모리 참조를 전송하고, 쓰기 연산과 관련해서 단계 캐시가 검색된 경우 쓰기 연산을 무시하며, 읽기 연산과 관련해서 단계 캐시가 검색된 경우 검색되거나 저장된 스레드 id와 함께 응답 수신 버퍼로부터 결과를 획득하도록 구성된다.
- [0033] 또 다른 실시형태에서, 상기 메모리 장치는 멀티 연산과 관련해서 스크래치패드를 액세스하고, 즉, 스크래치패드에 데이터를 저장하고 스크래치패드에서 데이터를 검색하며, 내부 프로세서 멀티연산 결과를 계산하기 위해 검색된 또는 저장된 스레드 id와 함께 이를 공유 메모리 시스템에 전송하도록 구성되며,
- [0034] 상기 메모리 장치는 최종 멀티연산 결과(멀티-프리픽스)를 결정하기 위해 공유 메모리 시스템으로부터 응답 수신 버퍼에서 수신된 데이터 및 상기 내부 프로세서 결과 모두를 이용하도록 더 구성될 수 있다.
- [0035] 또 다른 실시형태에서, 본원에서 설명되는 단계 캐시에 액세스하도록 구성되는 해시 및 구성 장치를 구비한다. 상기 해시 및 구성 장치는 바람직하게 액세스 포트를 통해, 더욱 바람직하게는 해시 및 구성 장치의 전용으로 구성되는 단일 액세스 포트를 통해 상기 단계 캐시에 결합된다. 상기 해시 및 구성 장치는 (예를 들어, 연산 레지스터에서) 프로세서에 의한 메모리 연산, (예를 들어, MAR 레지스터에 정해진) 메모리 주소, (예를 들어, MDR 레지스터에 정해진) 메모리 데이터, 스레드 id, 단계 카운터의 소정 수의 최하위 비트(LSB), 및 단계 캐시 액세스의 결과(즉, (검색이 성공인지 오류인지 여부에 따라 캐시 검색의 결과의 특성)로 이루어진 군에서 선택되는 적어도 하나의 요소를 기반으로 메모리 참조 메시지를 구성하도록 구성될 수 있다.
- [0036] 일부 실시형태에서, 상기 단계 캐시는 멀티-웨이 집합 연산(multi-way set associative) 단계 캐시일 수 있다.
- [0037] 일부 추가적인 또는 대안적인 실시형태에서, 상기 주소는 예를 들어 랜덤하게 선택된 해싱 함수를 이용하여 생성될 수 있다. 상기 HCU는 이후 메모리 참조에 대한 해시 주소를 계산하기 위해 이용된다.
- [0038] 또 다른 실시형태에서, 상기 메모리 장치는 본원에서 설명되는 스크래치패드에 액세스하도록 구성되는 메모리 요청 전송 로직 개체를 구비한다. 상기 스크래치패드는 액세스 포트를 통해, 바람직하게는 전송 로직 개체의 전용으로 구성될 수 있는 단일 액세스 포트를 통해 액세스될 수 있다.
- [0039] 상기 전송 로직 개체는 해시 및 구성 장치에 의해 제공된 메모리 메시지에 따라 공유 메모리 시스템에 메모리 참조를 전송하도록 구성될 수 있다. 메모리 연산을 기반으로, 상기 전송 로직은 멀티연산의 중간 결과 또는 내부 프로세서 결과를 결정하고, 바람직하게는 메모리 응답 대기 큐를 통해 응답 수신 버퍼에 대해 (신속한) 응답으로서 내부 메모리 (멀티)연산의 결과와 같은 내부 데이터를 제공하도록 구성될 수 있다.
- [0040] 실제로, 관련된 실시형태에서, 상기 메모리 장치는 소정 수의 파이프라인 단계와 관련된 응답 대기 큐를 더 포함하고, 상기 큐는, 예를 들어, 전송 논리로부터의 신속한 응답이 응답 수신 버퍼에 대해 적절한 순서로 전달되어 공유 메모리로부터 수신된 관련 메모리 참조를 적시에 검색한 후 수신 ALU를 이용하여 최종 연산 결과를 도출할 수 있도록, 메모리 요청 전송 로직 개체와 응답 수신 버퍼를 연결한다.
- [0041] 또 다른 양태에 따라, 에뮬레이션된 공유 메모리(ESM) 구조에서 인터리브드 인터-스레드 파이프라인을 구비하는 멀티-스레드화된 프로세서의 데이터 메모리의 참조를 처리하기 위한 방법이 제공되고, 상기 방법은,
- [0042] 멀티스레드화된 실행의 진행중인 단계의 마지막까지 데이터 어레이가 유효한 상태로 유지되는 연산 캐시 메모리 어레이를 형성하는 단계 캐시(504)를 획득하는 단계, 상기 메모리 어레이는 데이터 필드를 갖는 다수의 캐시 라인을 포함하고, 각각의 라인은, 바람직하게는 독점적으로, 주소 태그에 대한 데이터 필드 및 주소 태그에 의해 지정된 데이터 메모리 위치를 나타내는 제 1 스레드의 스레드 id에 대한 데이터 필드를 포함하고,
- [0043] 중간 결과와 같은 멀티연산의 내부 데이터를 저장하기 위한 메모리 버퍼를 형성하는 스크래치패드(506)를 획득하는 단계, 상기 버퍼는, 바람직하게는 독점적으로, 프로세서의 각각의 스레드에 대해 하나의 데이터 필드를 포함하고,
- [0044] 캐시 검색을 위해 상기 단계 캐시에 액세스하고, 프로세서에 의한 데이터 메모리의 참조를 포함하는 멀티연산의 실행시 프로세서 파이프라인의 다양한 클록 사이클 및 다양한 단계에서 상기 내부 데이터를 검색 및/또는 저장하기 위해 스크래치패드에 액세스하는 단계를 포함한다.

- [0045] 본 기술분야의 숙련자가 이해할 수 있는 바와 같이, 본원에 제시된 메모리 장치의 다양한 실시형태에 관한 다양한 고려가 필요한 부분만 약간 수정하여 방법의 실시형태에 유연하게 적용될 수 있고 반대의 경우도 마찬가지이다.
- [0046] 본 발명의 유용성은 이의 각각의 특정 실시형태에 따라 다수의 주제로부터 발생한다. 일반적으로, 하나의 메모리 주소에 대한 동시적인 메모리의 참조는, 각각의 프로세서에 의한 참조가 멀티스레드화된 실행의 단일 단계 동안 바람직하게 하나로 감소될 수 있도록 단계 캐시에 의해 구현된다. 멀티(-프리픽스)연산의 중간 데이터를 저장하기 위해 스크래치패드 메모리가 이용된다.
- [0047] 해당 액세스(파이프라인) 단계, 액세스 요소 및 내부에 저장된 데이터에 대해 본원에 개시된 바와 같은 단계 캐시와 스크래치패드의 특징 및 원래의 메모리 장치의 변경을 통해, 상당히 단순화된 하드웨어 구현이 달성되었고, 여기서 단계 캐시의 면적은 이전에 공개된 버전의 25% 이하로 축소할 수 있고, 스크래치패드의 면적도 그에 따라 감소되고, 단계 캐시와 스크래치패드 내의 제 2 포트는 제거되고, 많은 수의 비교기를 구비하는 복잡한 수신 응답 논리가 더욱 단순한 수신 버퍼로 교체되며, 다양한 클록 사이클 동안 변경된 단계 캐시와 스크래치패드를 모두 액세스함으로써 클록 사이클을 단축시킨다.
- [0048] "조정 수"라는 표현은 본원에서, 예를 들어, 하나, 둘, 또는 셋과 같이 하나(1)에서 시작하는 임의의 양의 정수를 나타낸다.
- [0049] "다수"라는 표현은 본원에서, 예를 들어, 둘, 셋, 또는 넷과 같이 둘(2)에서 시작하는 임의의 양의 정수를 나타낸다.
- [0050] "멀티연산"이라는 용어는 본원에서 일반적으로 실제 멀티-프리픽스 또는 그 밖의 해당 연산과 같이 그러나 멀티-프리픽스와는 대조적으로 프로세서에 다시 전송되는 반환값, 즉, 그에 따라 변경되는 대상 메모리 위치의 내용을 갖지 않는 연산 누적 연산을 칭한다.
- [0051] 본 발명의 다양한 실시형태가 첨부한 청구항에 개시된다.

도면의 간단한 설명

- [0052] 다음, 본 발명은 첨부한 도면을 참고로 더욱 상세하게 설명된다. 도면에서:
- 도 1은 실리콘 플랫폼 상에서 공유 메모리를 에뮬레이션하기 위한 실현 가능한 확장형 구조의 블록도이다.
- 도 2는 실현 가능한 ESM 구조, 본질적으로는 CMP ESM 구조의 또 다른 도면이다.
- 도 3은 MCRCW ESM 프로세서의 일 실시형태의 고차원 블록도 및 파이프라인이다.
- 도 4는 MCRCW ESM CMP 구조에서 프로세서의 메모리 장치의 종래 기술 버전을 도시하고 있다.
- 도 5는 본 발명에 따른 ESM 구조를 위한 메모리 장치의 일 실시형태를 도시하고 있다.
- 도 6은 본 발명에 따른 방법의 일 실시형태를 도시하고 있다.

발명을 실시하기 위한 구체적인 내용

- [0053] 본 발명의 일 실시형태에 따른 메모리 장치(MU)는 일반적으로, 예를 들어, 전용 명령어 메모리, 연산 단계 캐시(단계-인식 교체 정책에 의해 관리되는 멀티스레드화된 실행의 진행중인 단계의 마지막까지 내부에 삽입되는 데이터를 보유함) 및 프로세서에 부착된 비연상(스레드-어드레스) 스크래치패드 버퍼, 및 멀티-메시(multi-mesh) 상호 연결 네트워크와 같은 고대역 네트워크를 통해 프로세서(코어)에 결합되는, 물리적으로 분산되었지만 논리적으로 공유된 데이터 메모리를 포함하는 칩 멀티프로세서(CMP) 또는 멀티프로세서 시스템 온 칩(MP-SOC) 시스템을 구현하기 위해 ESM 컴퓨터 구조와 관련하여 사용될 수 있다. 네트워크는 충분히 높은 가능성으로 임의의 통신 패턴에 대해 충분한 처리량 및 상당한 지연이 달성될 수 있도록 프로세서를 분산 메모리 모듈에 연결한다.
- [0054] 코드의 읽기 전용 부분에 대한 처리량을 최대화하기 위해, 프로세서에서 메모리로 가는 참조 및 메모리에서 프로세서로 가는 응답에 대한 별도의 라인이 이용될 수 있다. 메모리 위치는 랜덤하게 또는 의사 랜덤하게 선택된 다항식에 의해 데이터 모듈에 걸쳐 분산될 수 있다. 구조는 바람직하게 고급 병렬 알고리즘에 대한 동시 메모리 액세스, 계산 프리픽스에 대한 멀티연산 및 선택적으로 상수 시간의 단축을 구현한다.
- [0055] 멀티연산은 다수의, 선택적으로, 두 개의 연속적인 명령어의 시퀀스로서 구현될 수 있다. 멀티-프리픽스와 같은

멀티연산을 실행하는 동안, 제 1 내부 프로세서의 멀티-프리픽스가 우선 결정될 수 있고, 그 결과 내부 프로세서의 멀티-프리픽스를 결정하기 위해 분산 공유 메모리 요소의 활성 메모리 모듈로 프로세서 방식의 결과가 전송될 수 있고, 이 내부 프로세서의 멀티-프리픽스를 기반으로 프로세서 자체 내에서 최종 멀티-프리픽스가 다시 결정될 수 있다.

[0056] 도 5를 참조하면, 본 발명의 일 실시형태에 따른 개선된 MU(500)가 도시되어 있고, 파이프라인 단계 동안 포함된 개체의 역할 이해를 용이하게 하기 위해 프로세서 파이프라인이 좌측에 도시되어 있다. MU(500)는 해시 및 구성 장치(HCU, 502), 메모리 참조 전송 로직 개체(510), 단일 어레이 단계 캐시(504), 스크래치패드(506), 응답 대기 큐(512b), 수신 버퍼(512) 및 수신 ALU(512c)를 포함한다. HCU(502)는 메모리 참조에 대한 해시 주소를 계산하고, MDR 레지스터에 정해진 메모리 데이터, MAR 레지스터에 정해진 메모리 주소, 스레드 식별자, 연산 레지스터로부터의 메모리 연산, 및 단계 카운터의 소정 수의 LSB를 기반으로 진행중인 메모리 참조 메시지를 구성할 뿐만 아니라 메모리 참조의 상태(멀티스레드화된 실행의 현재 단계 동한 임의의 스레드에 의해 이미 참조되었는지의 여부)를 결정하기 위해 단계 캐시에 액세스할 수 있다.

[0057] 연산을 근거로, 메모리 참조 전송 논리(510)는 스크래치패드(506)를 액세스하고 공유 메모리 시스템에 해당 참조를 전송하거나 또는 스크래치패드의 데이터를 이용하여 내부 메모리 연산 결과를 계산하고 그 결과를 (신속한) 응답으로서 메모리 응답 대기 큐(512b)에 제공한다. 응답 수신 버퍼(512)는 공유 메모리 시스템(508)으로부터 들어오는 응답 메시지를 수신하고 또한 동일한 개시자(스레드 id)를 갖는 응답이 이미 도착하였는지를 결정하기 위해 다단계 응답 대기 큐(512b)의 마지막 단계에 액세스된다. 그렇지 않은 경우, 응답이 도착할 때까지 파이프라인이 유보될 수 있다. 그렇지 않으면, 응답은 연산에 따라 수신 ALU(512c)에서의 추가 처리를 받는다. 응답 버퍼(512)는 멀티스레드화된 실행의 두 개의 연속하는 단계로부터 보류 비트와 데이터를 저장하기 때문에, 바람직하게 단독 또는 적어도 두 개의 데이터 어레이와 두 개의 포트를 포함한다.

[0058] 프로세서와 관련된 단계 캐시(504) 내의 각각의 라인은 개시자 및 주소 태그만을 포함할 수 있다. MU(500)가 단일 단계로부터의 단계 캐시(504)만을 액세스해야 하기 때문에, 단일 캐시 어레이와 하나의 액세스 포트가 충분하다. 프로세서가 메모리를 참조할 때마다, 단계 캐시 검색이 수행된다. 라인이 사용 중이고 주소가 해당 라인의 태그와 일치하는 경우 해당 캐시 라인에 대해 검색이 검출된다. 검색이 성공한 경우, 쓰기는 바로 무시되는 반면, 캐시로부터 개시자 데이터에 액세스함으로써 그리고 응답 수신 버퍼로부터 결과를 검색함으로써 읽기가 바로 완료된다(개시자 스레드에 의해 이미 이루어졌기 때문에 공유 메모리로 참조를 전송할 이유가 없다).

[0059] 검색 오류의 경우, 적용된 소정의 교체 정책을 사용하여 스레드 id가 캐시의 개시자 필드에 저장된다. 캐시 라인에 참조 정보를 저장하는 것과 동시에, 참조 그 자체가 공유 메모리 시스템(508)으로 전송되고, 보류 비트가 설정된다. 메모리(508)로부터 읽기 연산에 대한 응답을 수신하면, 데이터는 응답 대기 버퍼(512)의 데이터 필드로 입력되고, 해당 보류 필드는 지워진다. 멀티스레드화된 실행의 단계 간의 라인을 무효화하기 위해 소정의 캐시 붕괴 논리가 이용된다.

[0060] 프로세서와 관련된 스크래치패드(506)는 또한 제 2 포트가 제거되고 각각의 스크래치패드 라인이 프로세서의 각각의 스레드에 대해 하나의 데이터 필드를 가질 수 있도록 재설계되었다.

[0061] 내부 데이터가 스크래치패드(506)에 쓰여지고, 개시자가 단계 캐시(504)와 소위 개시자 레지스터(나머지 참조를 연결하기 위함, 도면에 도시되지 않음)에 저장되는 반면, 멀티연산에 대한 보류 비트가 스크래치패드(506) 대신에 응답 수신 버퍼(512)에 유지되고, 응답 데이터가 단계 캐시(504) 또는 스크래치패드(506) 대신에 응답 수신 버퍼(512)에 저장되며, 멀티연산의 종료 연산에 대한 응답 데이터가 단계 캐시(504) 또는 스크래치패드(506) 대신에 응답 수신 버퍼(512)에서 검색되도록 멀티연산은 바람직하게 두 개의 명령어의 시퀀스를 사용함으로써 구현된다(일례로, 표 2의 알고리즘 2 참조).

표 2

[0062]

알고리즘 2: 본 발명에 따른 MU의 일 실시형태에서 2-단계 MPADD 멀티연산의 구현

```

PROCEDURE Processor::Execute::BMPADD ( Write_data , Write_Address )
Search Write_Address from the StepCache and put the result in
matching_initiator
IF not found THEN
    StepCache[matching_initiator].address := Write_Address
    StepCache[matching_initiator].initiator_thread := Thread_id
    Initiator_thread:=Thread_id
ELSE Initiator_thread := StepCache[matching_index].Initiator_thread
On the next stage do
IF StepCache was hit during the last stage THEN
    Read_data := ScratchPad[Initiator_thread].data
    ScratchPad[Initiator_thread].data:= Write_data + Read_data
ELSE
Read_data:=0

PROCEDURE Processor::Execute::EMPADD ( Write_data , Write_Address )
Search Write_Address from the StepCache and put the result in
matching_initiator
IF not found THEN
    Initiator_thread:=Thread_id
ELSE Initiator_thread := StepCache[matching_index].Initiator_thread
On the next stage do
IF StepCache was not hit during the last stage THEN
    Send a EMPADD reference to the memory system with
    -- address = Write_Address
    -- data = ScratchPad[Initiator_thread].Data
    ReplyReceiveBuffer[Initiator_thread,Step].Pending:=True;

PROCEDURE Module::Commit_access::EMPADD ( Data , Address )
Temporary_data := Memory [Address]
Memory[Address] := Memory[Address] + Data
Reply_data := Temporary_data

PROCEDURE Processor::Receive_reply::EMPADD ( Data , Address , Thread )
Read_Data:= ReplyReceiveBuffer [Initiator_thread].Data+FastReply.Data
ReplyReceiveBuffer[Initiator_thread].Pending := False

```

[0063]

도 6은 본 발명의 일 실시형태에 따른 방법 흐름도를 오직 예로서 개시한다. 방법의 시작(602) 이후, 준비 작업이 수행된다, 즉, 제조되거나 또는 획득되는 호스팅(프로세서) 칩과 같은, 방법을 실행하기 위한 MU가 획득되고, 관련된 프로세서의 명령어 메모리에 저장되어 있는 원하는 유형의 프로그램 코드를 병렬 처리를 이용하여 실행하기 위해 나머지 ESM 및 지원 구조(전원 공급 등)와 함께 구성된다.

[0064]

특히, 멀티스레드화된 실행의 진행중인 단계의 마지막까지 데이터 어레이가 유효한 상태로 유지되는 연상 캐시 메모리 어레이를 형성하는 단계 캐시는 단계(604)에서 획득되고, 상기 메모리 어레이는 데이터 필드를 갖는 다수의 캐시 라인을 포함하고, 각각의 라인, 바람직하게는 독립적으로, 주소 태그에 대한 데이터 필드 및 주소 태그에 의해 지정된 데이터 메모리 위치를 나타내는 제 1 스레드의 스레드 id에 대한 데이터 필드를 포함한다.

[0065]

단계(606)에서, 중간 결과와 같은 멀티연산의 내부 데이터를 저장하기 위한 메모리 버퍼를 형성하는 스크래치패드가 획득되며, 상기 버퍼는, 바람직하게는 독립적으로, 프로세서의 각각의 스레드에 대해 하나의 데이터 필드를 포함한다.

[0066] 코드가 실행되면(도시된 파선 아래에 배치된 방법 단계), 코드에 존재하는 명령어가 인출되고 상기한 바와 같이 디코딩되고, 그 결과 단계 캐시 검색을 위해 캐시가 액세스되고(단계 610), 프로세서에 의한 데이터 메모리의 참조를 포함하는 멀티연산의 실행시 프로세서 파이프라인의 다양한 클록 사이클 및 다양한 단계에서 상기 내부 데이터를 검색 및/또는 저장하기 위해 스크래치패드가 액세스된다(단계 612). 공유 메모리에 대한 액세스, 참조에 대한 연산 및 결과 바이패스가 단계(614)에서 발생할 수 있다. 도시된 단계는 도면에 도시된 점선의 루프-백 화살표에 의해 나타낸 바와 같이 다양한 스레드 및 새로운 명령어를 처리하기 위해 반복적으로 실행될 수 있다. SEQ(618)는 명령어/프로그램 시퀀싱을 말한다.

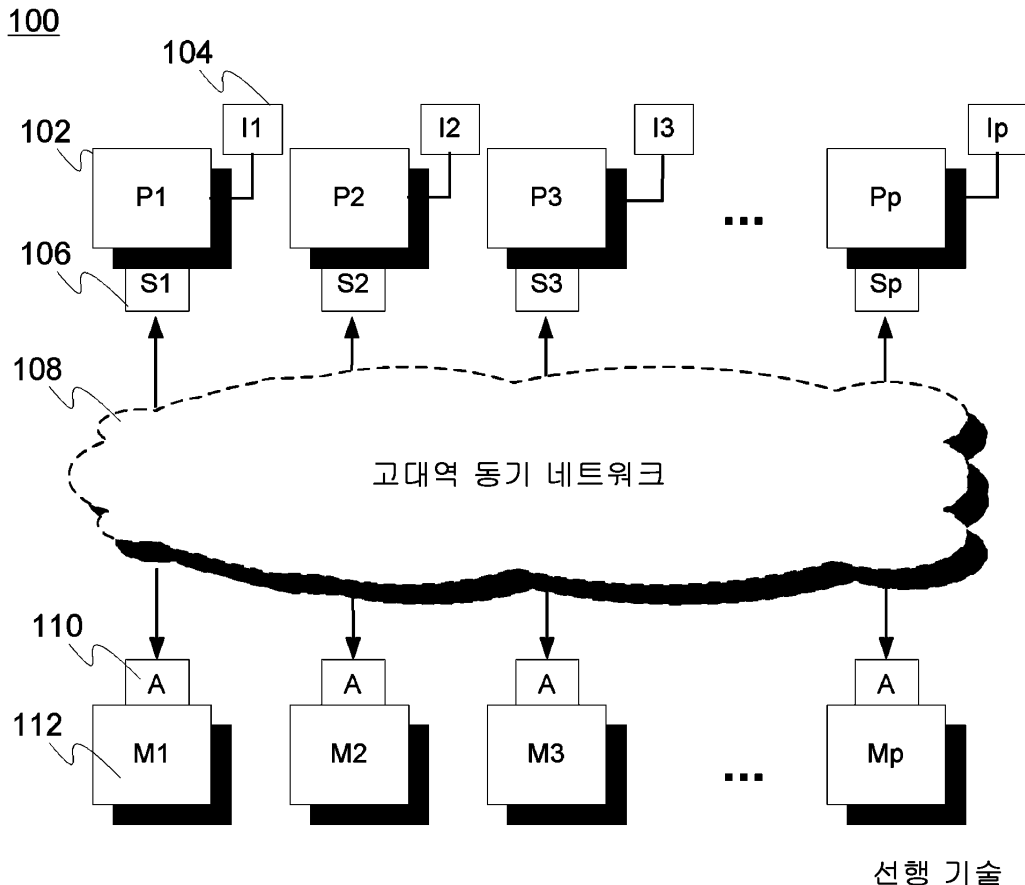
[0067] 단계(616)에서, 방법의 실행이 종료되고, 이는 일반적으로 프로그램 코드 실행의 종료와 함께 발생한다.

[0068] 상기 개시된 방법의 도면의 방법 단계의 상호간의 순서와 전체적인 존재는 각각의 특정 사용 시나리오에 의해 설정된 필요조건 및, 예를 들어, 실행중인 명령어와 관련된 메모리 연산에 의해 설정된 필요조건을 근거로 숙련자에 의해 변경될 수 있다.

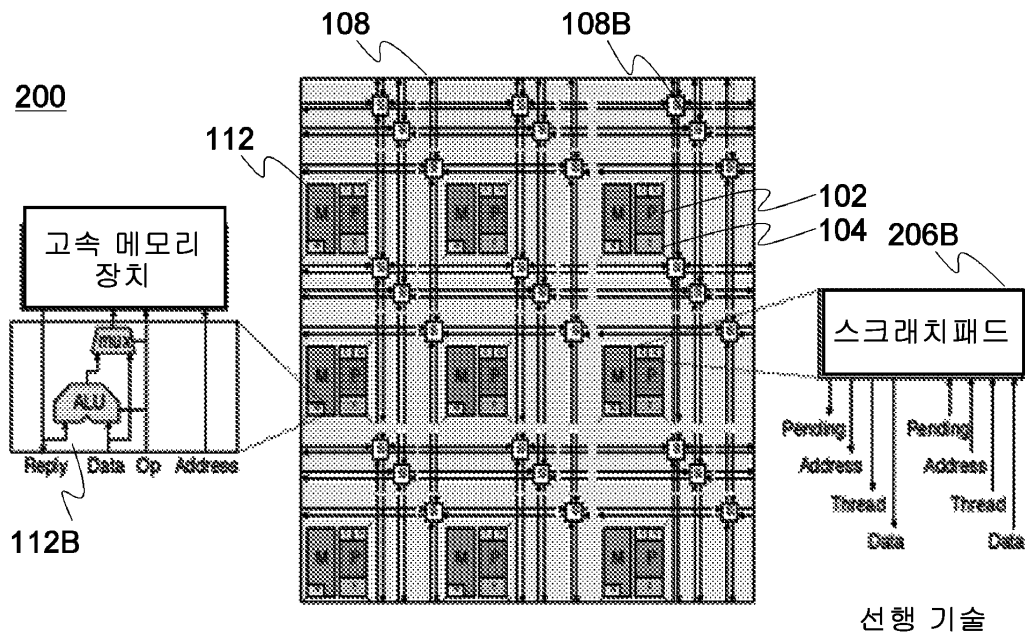
[0069] 따라서, 본 개시와 일반 상식을 근거로, 숙련자는 필요한 변경, 삭제, 및 부가와 함께 각각의 특정 용도에서 첨부한 청구항에 의해 정의된 본 발명의 범위를 구현하기 위해 제공된 교시를 적용할 수 있다. 일반적으로, 본원에 명시된 다양한 원리는 또한 본 기술분야의 숙련자가 이해하는 바와 같이 명확하게 개시된 것에서 벗어나는 프로세서 구조에서 이용될 수 있다.

도면

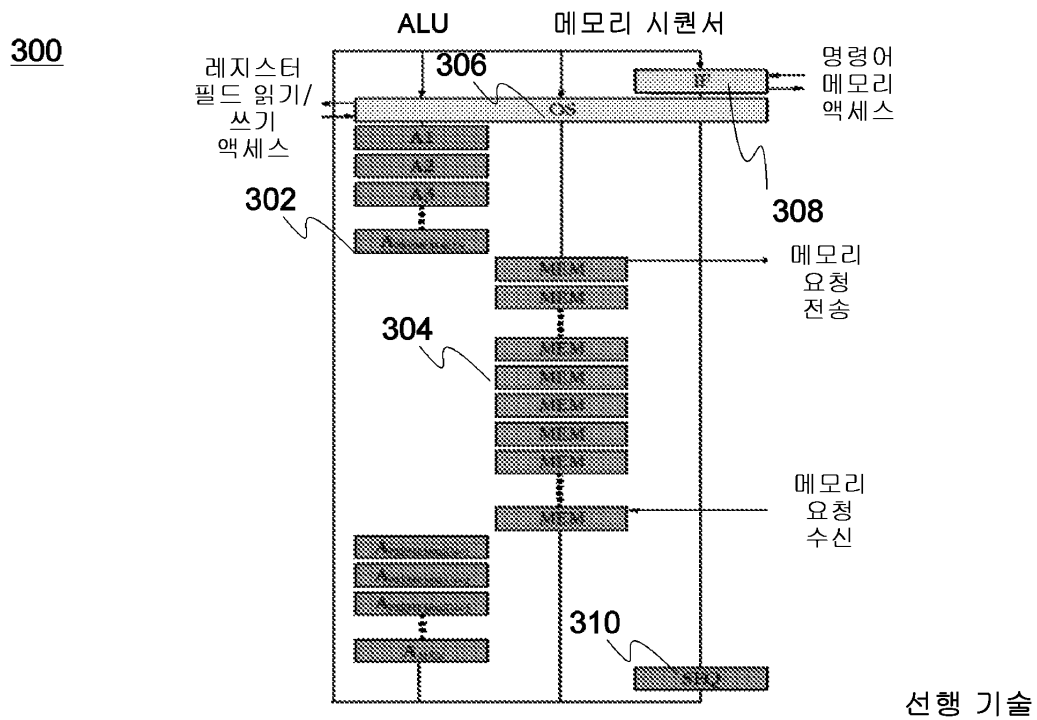
도면1



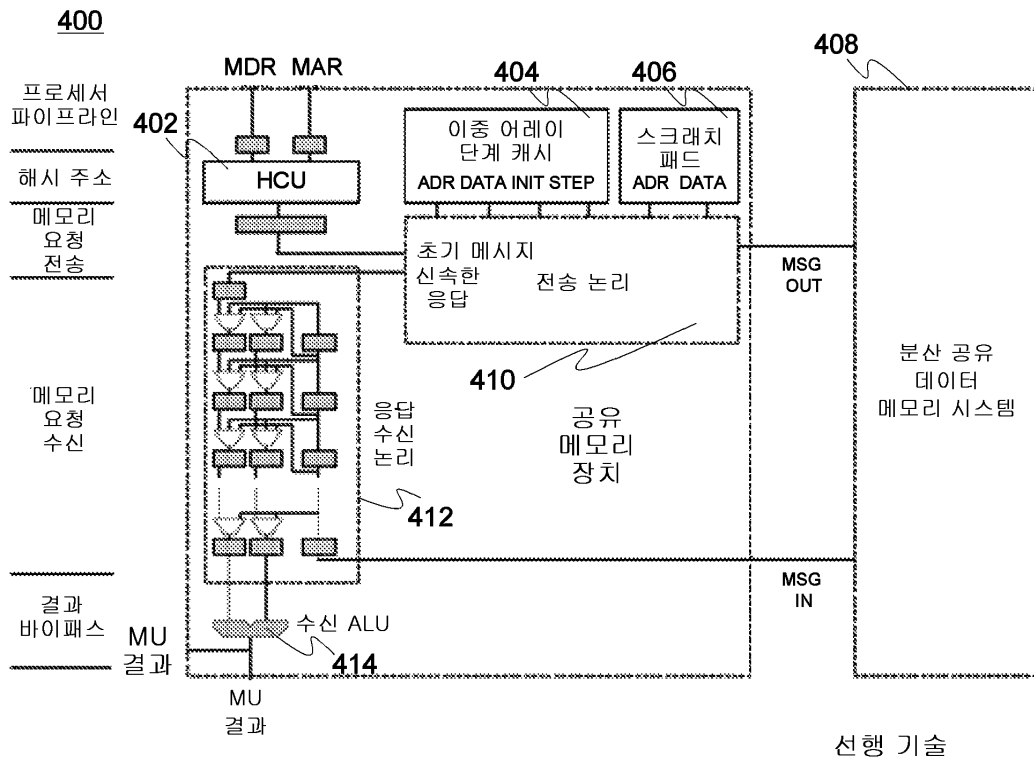
도면2



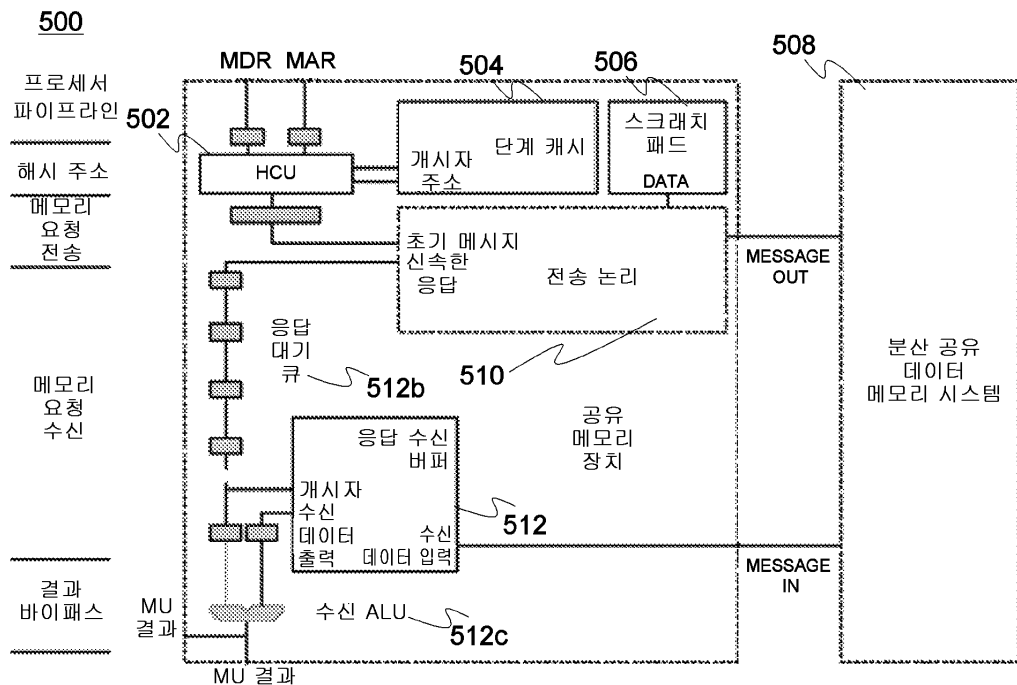
도면3



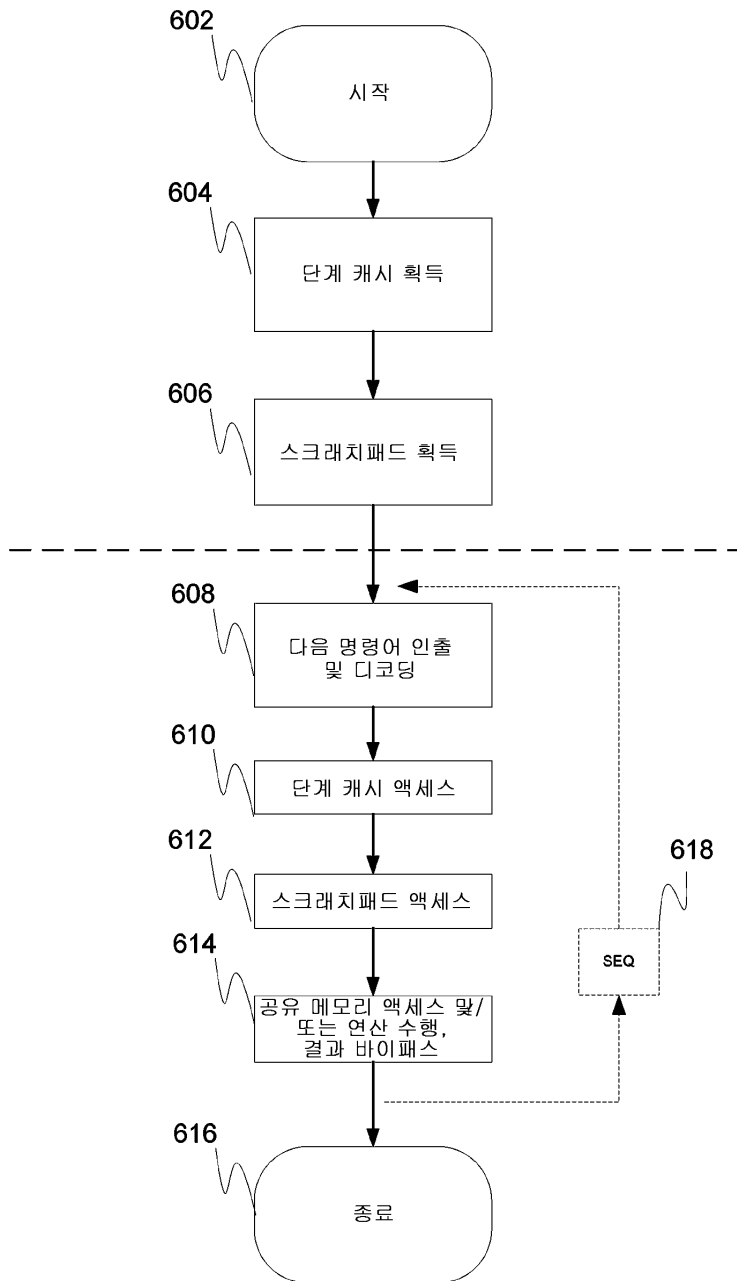
도면4



도면5



도면6



【심사관 직권보정사항】

【직권보정 1】

【보정항목】 청구범위

【보정세부항목】 청구항 13

【변경전】

제 12 항에 있어서,

상기 전송 논리 개체는 공유 메모리 시스템에 메모리 참조를 전송하고, 소정의 멀티연산의 중간 결과 또는 내부 프로세서 결과를 결정하며, 바람직하게는 메모리 응답 대기 큐를 통해 상기 응답 수신 버퍼(512)에 대해 로컬 응답으로서 내부 메모리 (멀티)연산의 결과와 같은 내부 데이터를 제공하도록 구성되는 것을 특징으로 하는 메모리 장치.

【변경후】

제 12 항에 있어서,

상기 전송 논리 개체는 공유 메모리 시스템에 메모리 참조를 전송하고, 소정의 멀티연산의 중간 결과 또는 내부 프로세서 결과를 결정하며, 바람직하게는 메모리 응답 대기 큐를 통해 응답 수신 버퍼(512)에 대해 로컬 응답으로서 내부 메모리 (멀티)연산의 결과와 같은 내부 데이터를 제공하도록 구성되는 것을 특징으로 하는 메모리 장치.